

Login help cards: build, implement and maintain

Oct 1, 2026 · @Sacha

Overview

We are building a reusable group of "help cards" with an optional message just beneath them. It goes on the login page first, can be placed anywhere else on the site, and admins can edit it without a developer. Expect to spend about 4–6 hours.

The design lives in code (two Single Directory Components, or SDCs). The content lives in Drupal (a custom block type built with Paragraphs). A few small Twig templates connect the two.

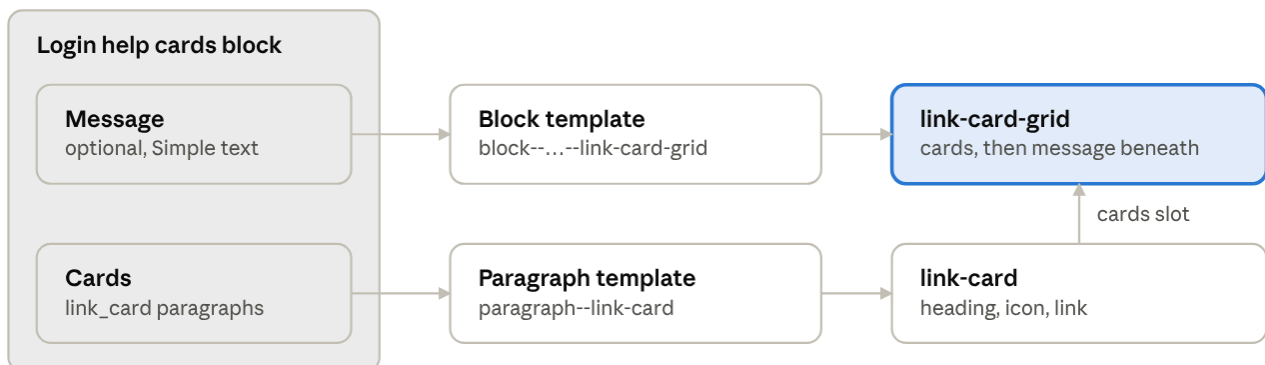
Layer	What it is	Where it lives	Who changes it
Components (SDC)	<code>link-card</code> (one card) and <code>link-card-grid</code> (the row of cards plus the message beneath)	<code>themes/custom/cycllinguk/components/</code>	Developers
Content model	Block type <code>link_card_grid</code> holding a message field and <code>link_card</code> paragraphs	Config in <code>config/sync</code>	Site builders (once)
Glue templates	Twig templates that pass field data into the components	<code>themes/custom/cycllinguk/templates/</code>	Developers
Content	The actual message and cards	Database	Editors
Placement	Where the block appears, such as <code>/user/login</code>	Block layout config	Site builders

Admins edit content; the theme components decide how it looks

CONTENT · admins edit

GLUE TEMPLATES · developers

SDC COMPONENTS · developers



How the parts connect · content, glue templates, components

Each card paragraph renders through `link-card`. The block template then puts those cards and the optional message into `link-card-grid`, which is what appears on the page.

Assumptions: the theme machine name is `cyclingunguk`, so swap in yours throughout. The site runs Drupal 11.x, where SDC is part of core and doesn't need enabling.

File map

```
themes/custom/cyclingunguk/
  components/
    link-card/
      link-card.component.yml
      link-card.twig
      link-card.css
      icons/  unlock.svg  faq.svg  partner.svg  phone.svg
      README.md
    link-card-grid/
      link-card-grid.component.yml
      link-card-grid.twig
      link-card-grid.css
  templates/
    block--block-content--type--link-card-grid.html.twig
    field--field-cards.html.twig
    field--field-message.html.twig
    paragraph--link-card.html.twig
```

Phase 0: Prerequisites (about 15 minutes)

Check these before you write any code.

1. **Check the Drupal version is 11.x** under *Reports* → *Status report*. SDC is built in, so there's no module to enable.
2. **Install Paragraphs**. This also enables Entity Reference Revisions.

```
composer require drupal/paragraphs
drush en paragraphs -y
```

3. **Turn on Twig debugging on your local site only**. Go to `/admin/config/development/settings` and tick "Twig development mode" and "Do not cache markup". The HTML source then shows which template rendered each part of the page, and which template names Drupal would accept instead.
4. **Prepare the four SVG icons**: padlock, questions, people with a cog, and phone. Use single-colour SVGs on a square canvas, such as 24×24. The colour inside the file doesn't matter, because the CSS recolours them.

Phase 1: Build the components (about 1–2 hours)

There are two components. `link-card` is one card. `link-card-grid` is the wrapper: the row of cards with an optional message underneath.

Step 1.1: `link-card/link-card.component.yml`

This file lists the inputs (props) the card accepts. The `icon` enum must match the `Icon` field's allowed values in Phase 2.

```
$schema:
https://git.drupalcode.org/project/drupal/-/raw/HEAD/core/assets/schemas/v1/m
etadata.schema.json
name: Link card
description: A single clickable card with heading, icon and link label.
status: stable
props:
  type: object
  required: [heading, label, url, icon]
  properties:
    heading:
      type: string
      title: Heading
    label:
      type: string
      title: Link label
    url:
      type: string
      title: URL
    icon:
      type: string
      title: Icon
      enum: [unlock, faq, partner, phone]
```

Step 1.2: `link-card/link-card.twig`

The whole card is one link. The icon is decorative, so screen readers skip it.

```
<a href="{{ url }}" class="link-card">
  <span class="link-card__heading">{{ heading }}</span>
  <span class="link-card__icon link-card__icon--{{ icon }}" aria-
hidden="true"></span>
  <span class="link-card__label">{{ label }}</span>
</a>
```

Step 1.3: link-card/link-card.css

Drupal attaches this automatically whenever the component renders. The icons use CSS masks, so they take the text colour.

```
.link-card {
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 0.75rem;
  padding: 1.25rem 1rem;
  background: #fff;
  border: 1px solid #dde3ea;
  border-radius: 8px;
  color: #3d4f66;
  text-decoration: none;
  text-align: center;
  transition: box-shadow 0.15s, transform 0.15s;
}
.link-card:hover { box-shadow: 0 4px 12px rgb(0 0 0 / 0.08); transform: translateY(-2px); }
.link-card:focus-visible { outline: 3px solid #f08a3c; outline-offset: 2px; }

.link-card__heading { font-weight: 600; }
.link-card__label { font-size: 0.875rem; }

.link-card__icon {
  width: 40px;
  height: 40px;
  background-color: currentColor;
  mask: no-repeat center / contain;
}
.link-card__icon--unlock { mask-image: url(icons/unlock.svg); }
.link-card__icon--faq { mask-image: url(icons/faq.svg); }
.link-card__icon--partner { mask-image: url(icons/partner.svg); }
.link-card__icon--phone { mask-image: url(icons/phone.svg); }
```

Step 1.4: link-card-grid/link-card-grid.component.yml

The grid has two slots. Slots take rendered Drupal output, such as fields, instead of plain strings.

```
$schema:  
https://git.drupalcode.org/project/drupal/-/raw/HEAD/core/assets/schemas/v1/m  
etadata.schema.json  
name: Link card grid  
description: A responsive row of link cards with an optional message beneath.  
status: stable  
slots:  
  cards:  
    title: Cards  
  message:  
    title: Message  
    description: Optional short notice shown beneath the cards.
```

Step 1.5: link-card-grid/link-card-grid.twig

The message box only appears when there's a message.

```
<div{{ attributes.addClass('link-card-group') }}>  
  <div class="link-card-grid">{{ cards }}</div>  
  {% if message %}  
    <div class="link-card-group__message">{{ message }}</div>  
  {% endif %}  
</div>
```

Step 1.6: link-card-grid/link-card-grid.css

```
.link-card-group {
  max-width: 760px;
  margin: 0 auto 2rem;
}
.link-card-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(160px, 1fr));
  gap: 1.5rem;
}
.link-card-group__message {
  margin-top: 1.25rem;
  padding: 0.875rem 1.125rem;
  background: #fff;
  border-left: 4px solid #f08a3c;
  border-radius: 4px;
  color: #3d4f66;
  text-align: center;
}
.link-card-group__message p:last-child { margin-bottom: 0; }
```

Step 1.7: Clear the cache and test in isolation

Drupal caches the list of components, so clear the cache whenever you add a new component folder.

```
drush cr
```

Temporarily paste this into any template, such as `page.html.twig`, and check the card renders. **Remove it** once you're happy.

```
{{ include('cyclingsuk:link-card', {
  heading: 'Test', label: 'Click me', url: '/', icon: 'phone'
}), with_context = false) }}
```

Phase 2: Build the content model (about 45 minutes, all in the admin UI)

This phase creates the forms editors will use. Do the steps in this order, because each one depends on the one before.

Step 2.1: Create a "Simple text" text format

Go to *Configuration* → *Text formats and editors* → *Add* (`/admin/config/content/formats/add`).

- Name: **Simple text**
- Editor: CKEditor 5, with only the **Bold**, **Italic** and **Link** buttons
- Leave roles unticked for now. Phase 5 grants it to editors.

This lets editors add a "Read more" link or some bold text, but not headings, images or tables that would break the layout.

Step 2.2: Create the paragraph type `link_card`

Go to *Structure* → *Paragraph types* → *Add* (`/admin/structure/paragraphs_type/add`).

Field	Type	Settings
<code>field_heading</code>	Text (plain)	Required. Max 40 characters
<code>field_link</code>	Link	Required. Internal and external links. Link text required
<code>field_icon</code>	List (text)	Required. Allowed values: <code>unlock Padlock</code> , <code>faq Questions</code> , <code>partner People</code> , <code>phone Phone</code>

The list keys (`unlock` , `faq` and so on) **must match** the `enum` in `link-card.component.yml` . This is the one place where content and code need to agree.

Step 2.3: Create the block type `link_card_grid`

Go to *Structure* → *Block types* → *Add* (`/admin/structure/block-content/add`).

1. Delete the default `body` field.
2. Add the fields:

Field	Type	Settings
<code>field_cards</code>	Entity reference revisions → Paragraph	Unlimited values. Reference type: <code>link_card</code> only
<code>field_message</code>	Text (formatted, long)	Optional. Allowed format: Simple text only

3. Under **Manage form display**, set `field_cards` to the **Paragraphs** widget with edit mode "Closed", which is easier to scan. Put `field_cards` above `field_message`, matching the page.
4. Under **Manage display**, hide the **label** on both fields.

Step 2.4: Create the first block

Go to *Content* → *Blocks* → *Add* (`/admin/content/block`) and choose "Link card grid".

- Block description: **Login help cards**
- Add the four cards:

#	Heading	Link text	URL	Icon
1	Trouble logging in?	Reset your password	<code>/user/password</code>	Padlock
2	Why the changes?	Read our FAQ	Your FAQ URL	Questions
3	Delivery partner?	Login here	Partner login URL	People
4	Need support	Contact us	Contact page URL	Phone

- Message: optional. Leave it blank for now to test the empty state.

Phase 3: Glue templates (about 30 minutes)

Four small templates pass Drupal's field data into the components. Only the block template contains any logic. The rest just strip out Drupal's default wrapper markup.

Step 3.1: `templates/paragraph--link-card.html.twig`

This renders each card paragraph through the `link-card` component.

```
{{ include('cycllinguk:link-card', {
  heading: paragraph.field_heading.value,
  label:   paragraph.field_link.title,
  url:     paragraph.field_link.0.url.toString(),
  icon:    paragraph.field_icon.value,
}, with_context = false) }}
```

Step 3.2: templates/block--block-content--type--link-card-grid.html.twig

This combines the cards and the message in the `link-card-grid` component. It passes the message only if one has been entered, so no empty box appears.

```
{% set block_entity = content['#block_content'] %}
{{ include('cyclingsuk:link-card-grid', {
  attributes: attributes,
  cards: content.field_cards,
  message: block_entity.field_message.value is not empty ?
  content.field_message : null,
}, with_context = false) }}
```

This template name hasn't been checked against our site yet. With Twig debug on, view the page source next to the block. Drupal lists every template name it will accept there, so if this one isn't listed, rename the file to one that is.

Step 3.3: templates/field--field-cards.html.twig

This outputs the cards without Drupal's extra wrapper `<div>` s, which would break the CSS grid.

```
{% for item in items %}{{ item.content }}{% endfor %}
```

Step 3.4: templates/field--field-message.html.twig

This does the same for the message.

```
{% for item in items %}{{ item.content }}{% endfor %}
```

Step 3.5: Clear the cache and check

```
drush cr
```

View the block, or a test page where it's placed. In the page source, the Twig debug comments should show your four templates and the two components being used.

Phase 4: Place it on the login page (about 15 minutes)

Place the block in a region above the main content, and limit it to the login pages.

1. Go to *Structure* → *Block layout* → your front-end theme.

2. Choose a region above the main content, often "Content" or "Highlighted". Click **Place block** and choose "Login help cards".
3. Configure the block:
 - Untick **Display title**.
 - Under **Pages**, choose "Show for the listed pages" and enter `/user/login` and `/user`. Logged-out visitors see the login form at `/user` too.
 - Under **Roles**, you can optionally restrict it to *Anonymous user*.
4. Set the weight so the block sits above "Main page content", which puts the cards above the login form.
5. Check the page **logged out**, in a private window.

Reusing it elsewhere: place the same block again in another region or on other paths. For a different set of cards, such as "Membership help", create a new Link card grid block with its own cards and message. It uses the same design.

Phase 5: Permissions for editors (about 10 minutes)

Editors need three block permissions and one text format permission. Grant them at *People* → *Permissions* (`/admin/people/permissions`) for the editor role.

Permission	Module	Why
Link card grid: Create new content	Custom Block	Make new card groups
Link card grid: Edit content	Custom Block	Change cards and the message
Access the Custom block library page	Custom Block	Find the blocks at <code>/admin/content/block</code>
Use the Simple text text format	Filter	Without this, the message field is greyed out or missing

Don't grant "Administer blocks". Editors can then change content without being able to move or delete block placements. Paragraphs inherit access from the block they're in, so no Paragraphs permissions are needed.

Test by logging in as a user with only the editor role.

Phase 6: Test before launch (about 30 minutes)

Tick these off on your local or staging site before deploying to production.

- ☐ **Matches the mockup** at desktop, tablet and mobile widths
- ☐ **Keyboard:** Tab through the cards. The orange focus ring shows and Enter follows the link
- ☐ **Screen reader:** VoiceOver (Cmd+F5) announces each card as one link, such as "Trouble logging in? Reset your password"
- ☐ **Empty message:** with no message entered, no empty orange box appears under the cards
- ☐ **Long message with a link:** wraps tidily, and the link can be reached with the keyboard
- ☐ **Anonymous cache:** edit a card or the message, then refresh in a private window. The change shows immediately
- ☐ **Links:** click every card, including external URLs
- ☐ **Edge cases:** a long heading wraps tidily, and five cards still lay out sensibly
- ☐ **Editor role:** a user with only the editor role can edit the block, including the message, but can't reach Block layout

Phase 7: Deploy (about 15 minutes)

Code and config are deployed. The cards themselves are content, so they have to be created on each site.

1. Export config locally:

```
drush cex -y
```

You should see new YAML files for the text format, paragraph type, block type, fields, form and view displays, block placement and permissions.

2. **Commit** the theme's `components/` and `templates/` files, the `config/sync` changes, and `composer.json` and `composer.lock`.

3. **On staging, then production:**

```
composer install
drush updb -y
drush cim -y
drush cr
```

4. **Create the block content on each site.** The block placement refers to the block by a unique ID (UUID), and the cards don't travel with config. In practice, either:

- place the block *after* the config import (simplest, about 5 minutes per site), or

- use the [Default Content](#) module if you want the cards scripted.

Travels with the code deploy	Must be created on each site
Components, templates, CSS, icons	The "Login help cards" block and its cards
Block type, paragraph type, fields	The message text
Text format, permissions	

Phase 8: Maintenance

Editors handle everyday content changes in one form. Developers only get involved for new icons or design changes.

Editor tasks (no developer needed)

All of these are done at *Content* → *Blocks* → "Login help cards" → Edit.

Task	How
Change a card's wording or link	Open the card in the Paragraphs widget and edit it
Reorder cards	Drag them in the Paragraphs widget
Add or remove a card	"Add Link card", or "Remove" on the card
Swap an icon	Choose a different option from the Icon dropdown
Add, change or remove the message	Edit the Message field. Leave it blank to hide the box

Developer tasks

Adding a new icon touches three places. Deploy the code and config together.

1. Add `icons/newname.svg` to the `link-card` folder.
2. Add `newname` to the `enum` in `link-card.component.yml`, and add a `.link-card__icon--newname` CSS rule.
3. Add `newname|Label` to the `field_icon` allowed values in the UI, then run `drush cex`.

Changing the look: edit the component's `.css` or `.twig` only. Every place the cards appear updates at once.

Moving the message above the cards: swap the order of the two `<div>` s in `link-card-grid.twig` and change `margin-top` to `margin-bottom` in the CSS. If admins should choose the position, add a list field (`above|below`) to the block type and pass it to the component as a prop that sets a modifier class.

Adding a style variant, such as compact cards: add an optional prop like `variant: { type: string, enum: [default, compact] }`, use it as a CSS modifier class, and pass it from the paragraph template.

Using a card outside the block: call `include('cyclinguik:link-card', {...})` from any template. That reuse is the main benefit of using SDC.

Housekeeping

- **Updates:** include `drupal/paragraphs` in your normal `composer update` routine. SDC is stable core, so there's nothing extra to track.
- **Documentation:** keep a short `README.md` in `components/link-card/` that lists the props and the three steps for adding an icon.
- **Ownership:** agree who owns the component code and which role owns the content.

Troubleshooting

Symptom	Likely cause	Fix
A card is missing, or there's an error in the log	Icon value doesn't match the <code>enum</code>	Make the field's allowed values and <code>link-card.component.yml</code> match
Component changes don't show	Cached component list or templates	<code>drush cr</code>
Cards stacked in one column	Default field template is adding wrapper <code><div></code> s	Check that <code>field--field-cards.html.twig</code> is being used (Twig debug)
No message, even though one is entered	Block template not being used	Check the template name against the Twig debug suggestions (Step 3.2)
Editors can't see the Message field	Missing text format permission	Grant "Use the Simple text text format" (Phase 5)

Time estimate and open questions

The whole build is about 4–6 hours of work.

Phase	Time
0–1: Setup and components	1.5–2.5 h
2–3: Content model and glue templates	1.25–1.75 h
4–7: Placement, permissions, testing, deploy	1–1.5 h
Total	about 4–6 h

Open questions to discuss

- ☐ What is the theme's real machine name? Replace `cyclInguk` throughout.
- ☐ Which URLs go in the FAQ, delivery partner and contact cards?
- ☐ Which role or roles should be able to edit the cards and the message?
- ☐ Should the message be formatted text with links, or plain text only?
- ☐ Is there an existing icon set, such as Font Awesome, to reuse instead of separate SVGs?
- ☐ Should the cards also appear on `/user/password` or the registration page?
- ☐ Will production cards be created by hand, or scripted with Default Content?